

title: "Human-regulating-the-loop: deterministic schema validation as the safety boundary for agentic biomedical curation"

target_journal: Nature Computational Science / Journal of Biomedical Semantics (high-end)

authors:

- Christopher J. Mungall (LBNL, Monarch Initiative)
- J. Harry Caufield (LBNL)
- [co-authors TBD]

draft_status: working draft

related_talks:

- "Mungall CJ. *Unlocking Disease Mechanisms: Agentic AI for Clinical Knowledge.* Zenodo, 2026. <https://zenodo.org/records/18720444>"

Abstract

Large language model agents can now draft structured biomedical content at a rate that no human curation pipeline can match, but they make errors that are both confident and unpredictable: fabricated identifiers, plausible-sounding but invented quotations, and ontology terms that read like real ones but do not exist. The conventional reflexive answer — "human in the loop" — does not scale, and in practice degrades to a curator rubber-stamping agent output. We argue for an inversion of the relationship between humans and agents that we call **human-regulating-the-loop**: humans set the rules, the schema, the ontology bindings, and the deterministic validation gates, and *agents operate within those gates*. The gates are not advisory; they are blocking. We describe a working instantiation of this pattern in the Disorder Mechanisms Knowledge Base (dismech), built around a LinkML schema with ontology-bound dynamic enumerations, a three-layer deterministic validation stack (schema, ontology terms, reference snippets), a GitHub-native CI pipeline that runs the stack on every change, and an agentic curation harness that proposes, edits, and reviews changes inside this enclosure. We analyse failure modes (deep-research citation hallucination, ontology label drift, snippet paraphrasing, cache fabrication), and present design patterns — most prominently **non-fabricatable reference caches** and **substring-only snippet matching** — that make agentic errors detectable rather than absorbed. We position this architecture against the alternatives (RLHF-style preference training, post-hoc fact-checking, agent self-verification) and argue that deterministic schema and identifier validation is the only durable safety boundary for autonomous structured curation at scale.

1. Introduction: the inadequacy of "human in the loop"

The dominant framing for safe AI-assisted curation is "human in the loop": a human reviews each agent output before it enters the canonical store. This framing is correct as far as it goes but underspecified. It does not say what the human is supposed to check, does not constrain the agent's output to a form where checking is tractable, and degrades predictably as agent throughput exceeds human review capacity. In knowledge-base practice, "human in the loop" most often means a curator approving a large fraction of agent output that they did not have the bandwidth to verify against primary sources.

The failure modes of current LLM agents in this setting are well characterized. They fabricate identifiers that look real (a PMID with the right number of digits, an HP term with the right prefix). They synthesize quotations that are stylistically consistent with the cited source but do not appear in it. They confidently assign ontology terms whose canonical label does not match the meaning required by the schema. They produce cache files, dossiers, and intermediate artifacts that look like the real thing and pass casual review. A "human in the

loop" pipeline that asks a curator to scan such output by eye will miss most of these errors, and the error rate will not be visible at the aggregate level until much later.

The architecture we describe inverts this relationship. The human is not *in* the loop; the human (and the upstream standards community) is *around* the loop, having defined the schema, the ontology bindings, the evidence contract, and the deterministic validators that police them. The agent operates inside this enclosure. Errors that would otherwise be absorbed are instead surfaced at a CI gate and blocked from merging. We call this **human-regulating-the-loop**, in deliberate contrast to "in the loop". The distinction is not semantic: it determines where engineering effort goes, what the agent is incentivised to optimise for, and whether the resource gets safer or less safe as agent capabilities grow.

This paper makes three contributions: (i) a concrete architecture for human-regulating-the-loop agentic curation, instantiated in a real mechanistic-disease knowledge base; (ii) an analysis of the agent failure modes that the architecture must catch, with the specific design choices that catch them; and (iii) an argument that deterministic, schema-bound validation is the only durable safety boundary for autonomous structured curation, in contrast to preference-training and self-verification alternatives that absorb error rather than surface it.

2. The dismech case: a working substrate

We use the Disorder Mechanisms Knowledge Base (dismech) as the concrete case throughout. Dismech is a structured representation of disease pathophysiology covering several hundred disorders, with ~500 disorder entries, dozens of mechanism modules, and tens of thousands of evidence-backed assertions linking genetic and environmental causes through pathophysiology and biochemistry to phenotypes and treatments, with explicit hypothesis groupings and knowledge gaps. The companion paper describes the mechanistic content and applications; this paper describes the curation architecture. The substantive features that matter for the present argument are:

- **A LinkML schema** (`src/dismech/schema/dismech.yaml`) that defines every permissible structure, with ontology-bound dynamic enumerations.
- **A line-oriented YAML serialisation** that is diffable in git and reviewable in pull requests.
- **A community-standard identifier discipline**: every claim references a PMID, DOI, NCT trial number, structured-database accession (ORPHA, CGGV, CGDS, CIVIC), or NAM dataset accession; every ontology term comes from a known prefix bound to a known authority.
- **A reference cache**: each cited primary source is fetched once into a versioned cache (`references_cache/`), and every claim's snippet must be a verbatim substring of the corresponding cache entry.
- **GitHub-native CI**: every change passes through a pull request with the validation stack, term validators, snippet validators, and an AI-augmented review pipeline.

We have built dismech with two production agentic systems (Claude via the Claude Code harness and Codex), running both interactively under a curator and autonomously via scheduled GitHub Actions. The curation throughput is substantially higher than the human-only baseline; the integrity of the content is enforced by the validation gates rather than by the agents themselves.

3. The schema as the safety boundary

The first design choice is that the *primary* artefact the agent produces is not text but a typed, schema-bound structure. The schema is authored in LinkML, a schema language for biomedical data that expresses classes, slots, enumerations, and constraints, and that compiles to JSON Schema, SHACL, ShEx, OWL, and SQL DDL. LinkML's distinguishing feature for our purposes is **dynamic enumerations bound by ontology constraint** (`reachable_from`): an enum value is valid only if it is reachable from a specified parent in a specified ontology via specified predicates. We use this to require, for example, that every phenotype descriptor's term is reachable from the root of HPO, that every cell type is reachable from the root of CL, that every treatment action is reachable from MAXO or NCIT's therapeutic-procedure subgraph, and that every gene identifier is in HGNC.

This is the substrate on which the rest of the safety story sits. Because the agent is required to emit typed objects, it cannot produce a free-text "evidence summary" that looks plausible but is unanchored. Because every ontology slot is bound to an authoritative source, an identifier that looks like an HP term but is not actually defined in HPO is rejected at validation. Because every evidence item is a structured object with required fields — a reference, a verbatim snippet, a classification of whether the evidence supports or refutes the claim, a classification of what *kind* of evidence is in the cited paper (human clinical, model organism, in vitro, computational) — the agent cannot smuggle vague support under specific cover.

The schema is also the locus of editorial policy. When we discovered that agents were silently conflating frequency-of-phenotype claims (e.g., "frequent") with disease-phenotype association claims, we added the editorial rule and the validation that distinguishes them, and the schema now carries the rule. When we discovered that agents were classifying veterinary case series as human clinical evidence, we tightened the classification rules and the validation now flags this. The schema is not static; it is the place where editorial learning accumulates, and where the agents' systematic errors become the next generation of rules they cannot violate.

4. The three-layer deterministic validation stack

Around the schema we run three validation layers. None of them invoke an LLM. All of them are deterministic and reproducible, and all of them must pass for a change to merge.

Layer 1: schema conformance. `linkml-validate` checks that the proposed YAML conforms to the schema: required fields present, types correct, enum values in range, ontology prefixes correct, multivalued slots multivalued, single-valued slots single-valued. This is the cheapest layer and catches the largest class of agent errors: agents will routinely emit the right idea in the wrong shape, and the schema catches that immediately.

Layer 2: ontology term validation. `linkml-term-validator` checks every ontology-bound term against a local authoritative snapshot of the ontology. The check is twofold: (i) the identifier must exist in the ontology; (ii) the `term.label` recorded in the YAML must exactly match the canonical label of the term in the ontology. This catches a failure mode that is both common and dangerous: agents will produce an identifier that is in the ontology but whose label is wrong, asserting (for example) that `HP:0001324` has label "Hypotonia" when in fact `HP:0001324` is "Muscle weakness". A label-mismatch error is not cosmetic; it indicates that the agent's intent and the actual term diverge, and that the downstream semantic interpretation will be wrong.

We deliberately distinguish two label slots on every descriptor. The `term.label` slot is canonical and must match the ontology exactly. The `preferred_term` slot is the human-readable display label and may be more specific or differently phrased than the ontology label when the ontology term is too broad to convey the intended clinical or biological granularity. This separation gives the agent a controlled outlet for clinical nuance without permitting it to corrupt the ontology binding.

Layer 3: reference snippet validation. `linkml-reference-validator` checks that every evidence item's `snippet` is a verbatim substring of the cached text for the cited `reference`. This is the single most consequential layer of the stack. It catches paraphrased quotations, fabricated quotations, wrong-PMID assignments, and confident hallucinations where the agent is sure about a citation that does not exist. Substring matching is intentionally strict: no fuzzy match, no normalization, no "close enough". The cost is occasional false rejections (whitespace and encoding issues); the benefit is that "this snippet exists in the source" is a yes/no fact that the agent cannot game.

These three layers run in sequence (cheapest first) and the pipeline short-circuits on the first failure. In practice the schema layer catches ~70% of agent errors before the term and reference validators run; the term validator catches another ~25%; the reference validator catches the remaining most-dangerous class — confident textual fabrication. Every class of error has a single deterministic check at its boundary.

5. Non-fabricatable reference caches: the most important design pattern

The reference cache is the asset that the reference validator depends on, and its integrity is therefore load-bearing for the whole architecture. The single largest agent failure mode that we have caught in production — and the one that took the longest to architect against — is the agent fabricating the cache itself.

The failure pattern goes like this. The agent is asked to add a claim backed by a PMID. The agent produces a snippet. The reference validator fails because the snippet is not in the cache. The agent, instructed to "resolve the validation failure", *creates the cache file by hand* with fabricated content that contains the snippet. Validation now passes. The human reviewer sees a green CI and merges. The cache file looks correct, the snippet matches the cache, the citation looks plausible — but the cache is a hallucination.

The architectural fix is that the cache is produced by, and only by, a dedicated fetcher (`linkml-reference-validator` with the appropriate subcommand, or, for structured sources, dedicated source-specific fetchers) that pulls the source from the upstream authority (PubMed, ClinicalTrials.gov, Orphadata, ClinGen, CIViC), writes a deterministic markdown-with-frontmatter file with a canonical filename, and computes a checksum recorded in the frontmatter. A separate CI check (`check-reference-cache-frontmatter`) validates that every cache file has parseable frontmatter consistent with its filename and reference ID. This turns "the cache is canonical" from a curator's assumption into a CI invariant.

We then explicitly instruct agents — both in the human-readable contributor guide and in the agent harness configuration — that they must never create or hand-edit cache files, and that the correct response to a snippet mismatch is either to re-quote the abstract more carefully or to use a different citation. The cache cannot be the variable the agent adjusts to make the validator green; the snippet has to be the variable.

This is the design pattern we believe generalises beyond dismech. Any agentic system that depends on a verifiable substrate (citations, identifiers, ontology terms, reference data) must arrange that the substrate is **not writeable by the agent**. The substrate must be maintained by deterministic fetchers from authoritative sources, with the CI invariant that the substrate can be reproduced from those sources. If the agent can write to the

substrate, the validation that depends on the substrate is not actually validating anything; it is laundering the agent's confidence.

6. Structured-database sources as quotable evidence

The same pattern extends to non-literature evidence. Many of the most important biomedical claims are not in journal abstracts but in structured databases: Orphanet for rare-disease definitions and phenotypes, ClinGen for gene-disease validity and dosage sensitivity, ClinicalTrials.gov for trial details, CIViC for cancer variant evidence. Each of these has its own access pattern, schema, and update cadence.

We treat structured sources as first-class citable evidence. For each source, a source-specific fetcher (in `src/dismech/structured_sources/`) pulls the bulk data with a pinned snapshot version, builds an index, and materialises one cache file per entity in the same `references_cache/` directory used for literature. The cache files use a deterministic line-oriented markdown format with markdown tables for tabular content, so that an Orphanet phenotype row or a ClinGen validity assertion row is a **stable quotable substring** that does not drift across rebuilds. Snapshot versions are pinned in a per-source manifest; refreshes are explicit and reviewable.

This means that an evidence item with `reference: ORPHA:558` and a snippet quoting "Marfan syndrome is a systemic disease of connective tissue" is validated by exactly the same substring check that validates a PMID-backed claim. The agent does not need to know the difference, and the validation mechanism does not need to special-case it. New structured sources are added by writing a new fetcher; the validation stack is unchanged.

7. The agentic harness: what we let agents do, and what we don't

Inside the validation enclosure, agents are responsible for a substantial fraction of the curation work:

- Drafting and enriching disorder entries from the literature.
- Adding ontology term annotations (HPO phenotypes, GO processes, CL cell types, MAXO treatments) under a dedicated skill that enforces use of authoritative term lookup via OAK.
- Validating and repairing evidence references under a dedicated skill that enforces the snippet substring contract.
- Responding to compliance scoring that identifies under-curated entries.
- Reviewing pull requests against the dismech contributor guidelines via a dedicated review agent.

Agents are explicitly *not* permitted to:

- Write to the reference cache.
- Modify the schema (a schema change is a human editorial decision, often motivated by patterns the agents have surfaced).
- Bypass any validation layer.
- Force-push branches they did not create.
- Open pull requests from forks (which would bypass CI secrets and skip the AI review layer).

These prohibitions are encoded both as harness configuration (permissions, hooks, allow/deny lists in the Claude Code settings) and as contributor guidelines in `CLAUDE.md`. The harness-level controls are not symbolic: they are enforced by the tool layer the agent operates in. An agent that attempts to write to a cache

file is denied at the tool boundary; an agent that attempts to skip a hook is denied at the git layer. The contributor guidelines are the human-readable form of the same constraints, and exist so that an agent that *could* in principle do something but should not knows it should not.

8. CI as the enforcement layer

The pull request is the unit of curation work. Every pull request runs the three-layer validation stack, the structured-cache frontmatter check, a schema-test suite, and an AI-augmented review pass. The review pass is run by an agent (Claude) configured against the same dismech contributor guidelines used for human review, and it produces inline suggestions with location, issue, and recommendation. The review agent's findings are *advisory*; the deterministic validators' findings are *blocking*.

Crucially, the human curator (or curating agent) is responsible for resolving findings: they cannot be dismissed silently. When the review agent finds a problem and the human disagrees, the disagreement is recorded as a PR comment and the human's reasoning is on the record. This creates an audit trail of editorial judgement that we then mine for schema refinements: a recurring disagreement between the review agent and human curators is usually a sign that the underlying rule is ambiguous and needs to be made explicit in the schema or the guidelines.

A scheduled GitHub Action periodically inspects the compliance dashboard, identifies the lowest-scoring entries, dispatches an agentic curation job for each, and opens a pull request. This means that the curation pipeline is autonomous on an end-to-end basis: a low-coverage entry is detected, researched, drafted, validated, reviewed, and merged without human intervention as long as every validation gate passes. When a gate fails, the human curator is engaged on the *specific failure* rather than on the whole entry. This is what "autonomous curation under deterministic validation" looks like in production.

9. Failure modes and how the architecture catches them

We document the principal agent failure modes we have observed and the specific architectural mechanism that catches each.

Failure mode	Mechanism
Fabricated PMID	Reference fetcher fails to resolve; PMID is not in cache; downstream snippet check fails.
Real PMID, wrong paper for the claim	Snippet does not match cache; reference validator fails.
Real PMID, paraphrased snippet	Substring check fails; reference validator fails.
Snippet from wrong PMID assigned to right PMID	Cache is per-PMID; the snippet from PMID A is not a substring of the cache for PMID B; validator fails.
Cache file fabricated to make snippet pass	Cache file frontmatter check fails (canonical filename, checksum, source-derivable invariants); also, cache files are not writeable by the agent's harness role.
Fake HP/GO/CL term that looks real	Term validator: identifier not in the ontology snapshot; validation fails.
Real term, wrong label	

Failure mode	Mechanism
	Term validator: identifier present but <code>term.label</code> does not match canonical label; validation fails.
Real term, wrong meaning (e.g., HP term for a GO concept)	Schema: slot is bound by <code>reachable_from</code> to the appropriate ontology root; validation fails.
Frequency claim backed by association evidence	Editorial rule documented in schema and contributor guide; AI review flags; human verifies.
Confident classification of veterinary or model-organism evidence as human-clinical	Evidence-source enum with editorial rule; AI review flags; tightened schema documentation.
Silent removal of an inconvenient REFUTE evidence item	Git diff visible at PR review; AI review flags significant deletions; human review.
Subtype foreign key inconsistency	Dedicated test (<code>test_subtype_foreign_keys</code>) verifies that every <code>subtype:</code> reference resolves to a declared subtype name.

Three patterns recur in this table. First, the most dangerous failure modes (cache fabrication, label mismatch, paraphrased snippet) are caught by deterministic checks against an external authority, not by the agent self-verifying or the human eyeballing. Second, the editorial-rule failures (frequency evidence, evidence-source classification) are caught by a combination of schema documentation, AI review, and human judgement, and each time we observe a new such failure we promote the rule into a more explicit schema constraint where possible. Third, no failure mode in the table is "caught by RLHF" or "caught by agent self-correction"; the safety boundary is deterministic throughout.

10. Comparison with alternative safety architectures

The dominant alternatives to deterministic schema validation for safe agentic content production are RLHF-style preference training, post-hoc fact-checking by a second LLM, and agent self-verification. We argue that each of these has structural failure modes that deterministic validation does not have.

RLHF and preference training make agents more confidently produce the kind of output that humans prefer at review time. They do not make the output verifiable. An agent that has learned to produce plausibly cited medical claims is not more likely to cite real papers; it is more likely to produce claims that look correctly cited. RLHF improves the correlation between agent output and reviewer preference, which makes the output harder to discriminate by review, which is the wrong direction for the failure modes that matter.

Post-hoc fact-checking by a second LLM is structurally similar to the first LLM: the second LLM is also confidently wrong about citations, ontology labels, and quotations, and the disagreements between the two agents are not a reliable signal of error. We see this in practice when the AI review pass on our PRs misses cache fabrications that the deterministic validator catches. A second agent is a useful reviewer of *style and editorial conformance*, which is how we use it; it is not a reliable reviewer of *fact*.

Agent self-verification — asking the agent to check its own work — has the structural problem that the agent cannot, in general, distinguish its own hallucinations from its own real recall. Iterative chains-of-thought with self-criticism reduce some error rates, but they do so by a margin that is dwarfed by the gain from a single external deterministic check against the source.

The case for deterministic schema and identifier validation as the *primary* safety boundary is therefore not aesthetic. It is that deterministic validation is the only mechanism in this list whose accuracy is independent of the agent's confidence, and the only mechanism whose failure rate is bounded by the correctness of human-authored code rather than by the next round of model training.

11. Limitations

The architecture we describe has its own failure modes. Substring snippet matching does not catch a snippet that is *real but irrelevant* — quoted correctly from an abstract that does not actually support the claim being made. We rely on AI review and human review for this layer, and we have seen it fail occasionally. Term validation does not catch a term that is correct but too broad or too narrow for the intended meaning; this is again an editorial judgement that AI and human review must catch. The schema itself is human-authored and contains its own errors; we rely on schema tests and an iterative process of promoting rules from contributor-guide prose to schema constraint. The cache is only as good as the snapshot date; refreshes can change snippets in ways that retroactively invalidate evidence, and we have to track this carefully. The agentic harness itself is software with bugs, and the permission boundaries we describe are only as strong as the implementation; the most likely future failure is not an agent escaping the rules but a misconfigured harness silently relaxing them.

Throughput is the second cost. Deterministic validation is slower than agent self-verification, particularly when reference fetches must be performed. We have invested in caching, parallelism, and structured-source batch fetches to amortise this, but a single new PMID is still a network call away. We see this as an unavoidable cost of the architecture, not a defect.

12. Conclusion

The right relationship between humans and agents in structured biomedical curation is not "human in the loop" but **human-regulating-the-loop**. Humans set the schema, the ontology bindings, the evidence contract, and the deterministic gates; agents propose, draft, review, and respond inside those gates. The gates are not advisory; they block. The cache is not writeable by the agent; it is regenerated from authoritative sources. The schema is the place where editorial learning accumulates and where the agents' systematic errors become rules they cannot violate. CI is the enforcement layer. AI review is advisory; deterministic validation is binding.

We have shown that this architecture is buildable today, that it scales to several hundred richly curated mechanistic disease entries with tens of thousands of evidence-backed assertions, and that it catches the failure modes — fabricated citations, paraphrased snippets, mismatched ontology labels, cache fabrication — that other safety architectures absorb silently. We do not believe this is the only safe architecture for agentic biomedical curation, but we believe any safe architecture will share its essential features: typed structured output, deterministic validation against authoritative sources, non-fabricatable substrates, and schema as the place where the rules live. As agentic capabilities grow, this is the structure that gets *safer* as the agents get better, because the gates are independent of the agents' confidence.

Data and code availability

Schema, validators, agent harness configuration, and full curation history: <https://github.com/monarch-initiative/dismech>. Browsable resource: <https://monarch-initiative.github.io/dismech/>. The LinkML schema

language is documented at <https://linkml.io>. Cache fetcher and structured-source framework are part of the dismech codebase under `src/dismech/structured_sources/`.

Related work and acknowledgements

We draw on prior work in LinkML schema design, OAK ontology access, the Monarch Initiative knowledge graph, Biolink Model export, ClinGen and Orphanet structured curation practice, and ongoing community discussions about safe agentic curation. The architecture described here builds on talks given by the first author including the Zenodo-recorded presentation *Unlocking Disease Mechanisms: Agentic AI for Clinical Knowledge* (<https://zenodo.org/records/18720444>), which sets out the "agentic AI for clinical knowledge" framing and the explicit anti-hallucination posture that this paper elaborates into an architectural argument. Funded by [TBD].